Study book

Topic 6: HTTP + REST, controllers, DTOs

ProjectHub learning track: React/TypeScript to C#/.NET backend. Week 2, topic 6 of 20.

Brief overview

What this lesson is about. The first real API. HTTP from the server side (method, URL, status code, headers, body), REST conventions (resources, verbs, idempotency), the status codes a frontend relies on (200/201/204/400/404/409/500), ASP.NET Core controllers (routing, model binding, `[ApiController]`, `ActionResult<T>`), **DTOs** and why entities never leave the API, JSON and **ProblemDetails**.
What we built. `ProjectsController` (list, get, create, update, delete, add task), request/response records in `Contracts/`, `UpdateAsync`/`DeleteAsync` + `ConflictException` (409) in the service, `Program.cs` registering the service as a **singleton**, and a `.http` file with 16 requests. All 16 were run against the real API.
Week 1 comes together: async service + the request's `CancellationToken` (topic 4), one shared service used in parallel -> the `lock` (topic 4), custom exceptions caught by type (topic 3), DTO records (topic 1), mapping with extension methods (topic 2).

Contents

Part	What is inside
1	HTTP and REST: resources, verbs, safe and idempotent
2	Status codes (the checkpoint), 400 vs 409
3	How a request reaches our code (the pipeline)
4	The controller: inheritance, primary constructor, implicit `this` (your questions), every action
5	The exception journey from service to controller (your request, with your screenshots)
6	DTOs and mapping
7	JSON, enums as strings, ProblemDetails, automatic 400
8	Program.cs and the singleton; service changes
9	What the API really returned (16 requests)
10	Debugging the pipeline in Rider
11	Your answers, corrections, interview Q&A
12	Summary, cheat sheet, checklist, what comes next

Part 1. HTTP and REST

Part	Request	Response
Line	method + URL: `POST /api/projects`	status code: `201 Created`
Headers	`Content-Type: application/json`	`Content-Type`, `Location: /api/projects/4`
Body	JSON the client sends	JSON the server returns

In React you wrote the client (`fetch(...)`, `if (!res.ok)`); now you write the server and decide `res.status`. **REST:** resources are plural nouns in the URL; the HTTP method is the verb.

Method	URL	Meaning	Safe	Idempotent
GET	`/api/projects`, `/api/projects/5`	list / read one	yes	yes
POST	`/api/projects`	create (server picks the id)	no	**no**
PUT	`/api/projects/5`	replace the editable fields	no	yes
PATCH	`/api/projects/5`	change some fields (not used yet)	no	not necessarily
DELETE	`/api/projects/5`	delete	no	yes
POST	`/api/projects/5/tasks`	create a task inside project 5 (nested resource)	no	no

- **Safe:** does not change data. **Idempotent:** once or five times leaves the **same final state** (DELETE twice: deleted; the second call returns 404, the state is the same).
- **Why it matters:** after a network timeout a client can safely **retry** PUT and DELETE; retrying a POST may create duplicates.
- Not REST-y: `POST /api/createProject`, `GET /api/deleteProject?id=5` (verbs in URLs, a GET that changes data).

Common mistake

You first named 'put/patch' as idempotent. Correct: **GET, PUT, DELETE**; PATCH not necessarily (e.g. 'add 1 to a counter'); POST not. Re-check: right.

Part 2. Status codes (the checkpoint)

Code	Name	When	In ProjectHub
200	OK	success with a body	GET, PUT
201	Created	a new resource; add a **Location** header	POST project, POST task
204	No Content	success, nothing to return	DELETE
400	Bad Request	the request is invalid by itself (shape **or** values)	empty name, missing `name`, broken JSON
401 / 403	Unauthorized / Forbidden	not logged in / not allowed	topic 16
404	Not Found	the resource (or the route) does not exist	unknown id; `/api/projects/abc`
409	Conflict	valid request that conflicts with the **current state**	name already taken

Code	Name	When	In ProjectHub
500	Internal Server Error	an **unexpected** failure: our bug or infrastructure	unhandled exception (topic 12)

4xx = the client must change something; 5xx = the server has a problem — topic 3's expected vs unexpected errors, as HTTP.

Precisions from your quiz

400 vs 409: `"  "` is wrong whatever the database contains -> 400; retrying can never work. `"Mobile app"` is a valid name that is **taken right now** -> 409; it could work after the other project is renamed.
404 has two flavors: no route matches (`/api/projects/abc`, empty body) vs the resource does not exist (`/api/projects/999`, ProblemDetails).
500 means an unexpected failure; a global handler (topic 12) only *writes* the 500 response.

Part 3. How a request reaches our code

```
HTTP request -> Kestrel (web server) -> middleware (Program.cs: HTTPS redirect, auth, ...)
             -> routing: which action matches "POST /api/projects"?          (no match -> 404, empty body)
             -> model binding: JSON body -> CreateProjectRequest, "5" -> int id
             -> [ApiController] validation: invalid -> automatic 400           (the action never runs)
             -> your action -> result (Ok, NotFound, ...) -> JSON -> HTTP response
```

Express analogy: `app.use(...)` middlewares, then `router.post('/api/projects', handler)`; in ASP.NET the router reads **attributes** on the controller.

Part 4. The controller

4.1 The class line (your question)

```
public class ProjectsController(IProjectService projects) : ControllerBase
//                              └──────────── 1 ────────────┘   └──── 2 ────┘
```

1. **Primary constructor** (topic 3): `projects` is usable in every method. ASP.NET creates the controller **per request** and passes the registered `IProjectService` (dependency injection); nobody writes `new ProjectsController(...)`.
2. `: ControllerBase` = **inheritance** (TS `extends`): the controller gets `Ok()`, `NotFound()`, `NoContent()`, `CreatedAtAction()`, `Problem()`, `HttpContext`, `Request`, `Response`, `User`, `ModelState`.

```
// TypeScript equivalent
class ProjectsController extends ControllerBase {
  constructor(private projects: IProjectService) { super(); }
}
```

- The same `:` also means **implements an interface** (`ProjectService : IProjectService`). One base class, many interfaces; the base class comes first; the `I` prefix tells which is which.
- Framework base classes (`Exception`, `ControllerBase`) are the common exception to 'prefer composition over inheritance'.

4.2 Calling Ok() without this. (your question)

Yes: inside a class, a bare name is looked up on the current object, including inherited members. `Ok(x)` is exactly `this.Ok(x)`. In JS/TS `this.` is required (`ok(x)` would look for a variable).

- Lookup order: **locals/parameters** -> **class members** (own + inherited) -> outer scopes (static members, `using` namespaces).
- `this.` is needed when a parameter shadows a field (`this.name = name;`) — one reason for the `_camelCase` field convention.
- **Trap:** a primary constructor parameter is **not a field**: `this.projects` is a compile error; `projects` is simply in scope.

4.3 The whole controller

src/ProjectHub.Api/Controllers/ProjectsController.cs

```
using Microsoft.AspNetCore.Mvc;
using ProjectHub.Api.Contracts;
using ProjectHub.Api.Exceptions;
using ProjectHub.Api.Services;

namespace ProjectHub.Api.Controllers;

// [ApiController]: automatic 400 for invalid request bodies, ProblemDetails for error responses,
// and [FromBody]/[FromRoute] inferred from the parameter types.
[ApiController]
[Route("api/projects")]
public class ProjectsController(IProjectService projects) : ControllerBase
{
    // GET /api/projects -> 200 with a list of summaries
    [HttpGet]
    public async Task<ActionResult<IReadOnlyList<ProjectSummaryResponse>>> GetAll(CancellationToken cancellationToken)
    {
        var all = await projects.GetAllAsync(cancellationToken);
        return Ok(all.Select(p => p.ToSummaryResponse()).ToList());
    }

    // GET /api/projects/5 -> 200 with the project and its tasks, or 404
    [HttpGet("{id:int}")]
    public async Task<ActionResult<ProjectResponse>> GetById(int id, CancellationToken cancellationToken)
    {
        var project = await projects.GetByIdAsync(id, cancellationToken);
        return project is null ? NotFound() : Ok(project.ToResponse());
    }

    // POST /api/projects -> 201 Created + Location header, or 400 / 409
    [HttpPost]
    public async Task<ActionResult<ProjectResponse>> Create(CreateProjectRequest request, CancellationToken cancellationToken)
    {
        try
        {
            var project = await projects.CreateAsync(request.Name, request.Description, cancellationToken);
            return CreatedAtAction(nameof(GetById), new { id = project.Id }, project.ToResponse());
        }
        catch (ConflictException ex)
        {
            return Problem(detail: ex.Message, statusCode: StatusCodes.Status409Conflict);
        }
        catch (BusinessRuleException ex)
        {
            return Problem(detail: ex.Message, statusCode: StatusCodes.Status400BadRequest);
        }
    }

    // PUT /api/projects/5 -> 200 with the updated project, or 400 / 404 / 409
    [HttpPut("{id:int}")]
    public async Task<ActionResult<ProjectResponse>> Update(int id, UpdateProjectRequest request, CancellationToken cancellationToken)
    {
        try
        {
            var project = await projects.UpdateAsync(id, request.Name, request.Description, cancellationToken);
```

(continued)

```
            return Ok(project.ToResponse());
        }
        catch (NotFoundException ex)
        {
            return Problem(detail: ex.Message, statusCode: StatusCodes.Status404NotFound);
        }
        catch (ConflictException ex)
        {
            return Problem(detail: ex.Message, statusCode: StatusCodes.Status409Conflict);
        }
        catch (BusinessRuleException ex)
        {
            return Problem(detail: ex.Message, statusCode: StatusCodes.Status400BadRequest);
        }
    }

    // DELETE /api/projects/5 -> 204 No Content, or 404
    [HttpDelete("{id:int}")]
    public async Task<IActionResult> Delete(int id, CancellationToken cancellationToken)
    {
        try
        {
            await projects.DeleteAsync(id, cancellationToken);
            return NoContent();
        }
        catch (NotFoundException ex)
        {
            return Problem(detail: ex.Message, statusCode: StatusCodes.Status404NotFound);
        }
    }

    // POST /api/projects/5/tasks -> 201 Created, or 400 / 404
    [HttpPost("{id:int}/tasks")]
    public async Task<ActionResult<TaskResponse>> AddTask(int id, CreateTaskRequest request, CancellationToken cancellationToken)
    {
        try
        {
            var task = await projects.AddTaskAsync(id, request.Title, cancellationToken);
            // There is no "GET one task" endpoint yet, so Location points to the project that now contains it.
            return CreatedAtAction(nameof(GetById), new { id }, task.ToResponse());
        }
        catch (NotFoundException ex)
        {
            return Problem(detail: ex.Message, statusCode: StatusCodes.Status404NotFound);
        }
        catch (BusinessRuleException ex)
        {
            return Problem(detail: ex.Message, statusCode: StatusCodes.Status400BadRequest);
        }
    }
}
```

Piece	Meaning
`[ApiController]`	automatic 400 for invalid **requests**; ProblemDetails error bodies; binding-source inference (complex type -> body)
`[Route("api/projects")]`	base URL of every action
`[HttpGet("{id:int}")]`	GET `api/projects/{id}`; `:int` is a **route constraint**: `abc` matches no action -> 404 with an empty body
`int id`, `CreateProjectRequest request`	**model binding**: from the URL; from the JSON body (camelCase names, case-insensitive)
`CancellationToken cancellationToken`	filled with `HttpContext.RequestAborted`: cancelled when the client disconnects; passed down (topic 4)
`ActionResult<ProjectResponse>`	either the DTO (200) or another result; `<T>` documents the success type for OpenAPI. `Delete` uses plain `IActionResult` (never a body)
`CreatedAtAction(nameof(GetById), new { id }, body)`	**201** + `Location: http://localhost:5064/api/projects/4` (built from the GetById route) + body
`Problem(detail:, statusCode:)`	ProblemDetails body with our message
the repeated try/catch	deliberate for now; topic 12 moves it into **one global exception handler**

Part 5. The exception journey: service -> controller

Your note: 'we can catch those custom exceptions in `Update` because we threw them before'. Exactly: they are **thrown deep in the service** and **caught by type in the controller**.

The controller action:

```
// PUT /api/projects/5 -> 200 with the updated project, or 400 / 404 / 409
[HttpPut("{id:int}")]
public async Task<ActionResult<ProjectResponse>> Update(int id, UpdateProjectRequest request, CancellationToken cancellationToken)
{
    try
    {
        var project = await projects.UpdateAsync(id, request.Name, request.Description, cancellationToken);
        return Ok(project.ToResponse());
    }
    catch (NotFoundException ex)
    {
        return Problem(detail: ex.Message, statusCode: StatusCodes.Status404NotFound);
    }
    catch (ConflictException ex)
    {
        return Problem(detail: ex.Message, statusCode: StatusCodes.Status409Conflict);
    }
    catch (BusinessRuleException ex)
    {
        return Problem(detail: ex.Message, statusCode: StatusCodes.Status400BadRequest);
    }
}
```

```
0+1 usages  Claude
public async Task<Project> UpdateAsync(int id, string name, string? description, CancellationToken cancellationToken = default)
{
    var trimmedName = ValidateName(name);

    await SimulateDatabaseCallAsync(cancellationToken);

    lock (_gate)
    {
        var project = FindById(id) ?? throw new NotFoundException(nameof(Project), id);

        // The project may keep its own name; it may not take a name used by ANOTHER project.
        EnsureNameIsFree(trimmedName, exceptProjectId: id);

        project.Name = trimmedName;
        project.Description = description;
        return project;
    }
}
```

Your screenshot: ProjectService.UpdateAsync, the throwing lines underlined.

```
// Call only while holding _gate. The business rule: project names are unique, ignoring case.
2 usages  Claude
private void EnsureNameIsFree(string name, int? exceptProjectId)
{
    if (_projects.Any(p :Project => p.Id != exceptProjectId
                            && string.Equals(p.Name, name, StringComparison.OrdinalIgnoreCase)))
    {
        throw new ConflictException($"A project named '{name}' already exists.");
    }
}
```

Your screenshot: EnsureNameIsFree throws ConflictException.

What happens for PUT /api/projects/4 with {"name": "Mobile app"}

```
ProjectsController.Update             <- catch (ConflictException ex) -> 409
  └─ await projects.UpdateAsync(...)  <- exception re-thrown here (topic 4)
       └─ lock (_gate) { ... }        <- lock released automatically on the way out
            └─ EnsureNameIsFree(...)  <- throw new ConflictException(...)   starts here
```

1. `EnsureNameIsFree` throws: execution stops on that line; the method exits immediately.
2. The exception travels **up the call stack**, leaving every method, until a `catch` of a matching type.
3. It passes the `lock`, which is **released automatically** (`lock` is a `try { } finally { release }`); otherwise one failed update would block all other requests.
4. `UpdateAsync` is async: the exception is stored in its Task and **re-thrown at the** `await` in the controller.
5. The catch blocks are checked **top to bottom by type**: `ConflictException` matches -> `Problem(..., 409)`.

Thrown in the service	Where	Caught in Update	HTTP
`NotFoundException`	`FindById(id) ?? throw new NotFoundException(...)`	`catch (NotFoundException ex)`	**404**
`ConflictException`	`EnsureNameIsFree` -> `throw new ConflictException(...)`	`catch (ConflictException ex)`	**409**
`BusinessRuleException`	`ValidateName`: empty or too long	`catch (BusinessRuleException ex)`	**400**
anything else (a bug)	anywhere	no matching catch	**500** (topic 12)

- **Catch order:** `ConflictException` **inherits** from `BusinessRuleException`, so a `catch (BusinessRuleException)` first would turn every conflict into 400. The compiler refuses that order (CS0160).
- **Why custom exception types (topic 3):** the controller asks 'what type is this?' instead of parsing message text; the type decides the status code, the message goes into `detail`.
- TS analogy: `throw new ConflictError(...)` deep in a service; `if (e instanceof ConflictError) return res.status(409)` in the Express handler.

Part 6. DTOs: the API never returns entities

Request DTOs (input)	Response DTOs (output)
`CreateProjectRequest(string Name, string? Description)`	`ProjectSummaryResponse(Id, Name, Description, CreatedAt, TaskCount, DoneCount)` — list items
`UpdateProjectRequest(string Name, string? Description)`	`ProjectResponse(Id, Name, Description, CreatedAt, Tasks)` — one project with tasks
`CreateTaskRequest(string Title)`	`TaskResponse(Id, ProjectId, Title, Status, CreatedAt)`

Why DTOs (have five ready for the interview)

1. **Over-posting / mass assignment:** a request DTO contains only what the client may set; nobody can send `"id": 999` or `"createdAt"`.
2. **Stable contract:** the JSON shape does not change when the database or entities change.
3. **The right shape per use case:** counts for the list, tasks for the detail view.
4. **No leaks (topics 3 and 5):** a DTO is a **new object**; nobody outside the service gets a reference to the stored entities.
5. **No accidental exposure** (a future `PasswordHash`) and **no serialization cycles** (Project -> Task -> Project).

Remember

Your answer: 'HTTP boundary; no returned references that could be mutated; safe copies; no exposure of e.g. a user's password'. Good. Add **over-posting** and **stable contract**.

src/ProjectHub.Api/Contracts/ProjectMappings.cs (extension methods, topic 2)

```
using ProjectHub.Api.Entities;

namespace ProjectHub.Api.Contracts;

// Entity -> DTO. Extension methods (Topic 2), so a controller can write project.ToResponse().
// The DTOs are NEW objects: callers of the API never get a reference to the stored entities.
public static class ProjectMappings
{
    public static ProjectSummaryResponse ToSummaryResponse(this Project project) => new(
        project.Id,
        project.Name,
        project.Description,
        project.CreatedAt,
        TaskCount: project.Tasks.Count,
        DoneCount: project.Tasks.Count(t => t.Status == TaskItemStatus.Done));

    public static ProjectResponse ToResponse(this Project project) => new(
        project.Id,
        project.Name,
        project.Description,
        project.CreatedAt,
        project.Tasks.Select(t => t.ToResponse()).ToList());

    public static TaskResponse ToResponse(this TaskItem task) => new(
        task.Id,
        task.ProjectId,
        task.Title,
        task.Status,
        task.CreatedAt);
}
```

Explicit mapping is easy to read and debug; libraries like AutoMapper or Mapster map by convention. Follow your team's choice.

Part 7. JSON, enums, ProblemDetails, automatic 400

- **System.Text.Json web defaults:** camelCase property names (`createdAt`), ISO 8601 dates (`"2026-09-01T09:00:00Z"`, `Z` = UTC).
- **Enums as strings:** `JsonStringEnumConverter` makes `"status": "Done"`. Without it (your debug step 6): `"status": 2` (Todo = 0, InProgress = 1, Done = 2): magic numbers for the frontend, and inserting a new enum value shifts the numbers and silently breaks old clients.
- **ProblemDetails** (RFC 9457, `application/problem+json`): the standard error body with `type`, `title`, `status`, `detail`, `traceId`; the frontend reads every error the same way; `traceId` finds the request in logs (topic 17).
- **Automatic 400:** with nullable reference types on, `string Name` (non-nullable) is **required**; missing -> 400 with an `errors` dictionary **before the action runs**. Malformed JSON: the same.

```
{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.10","title":"Conflict","status":409,
 "detail":"A project named 'WEBSITE REDESIGN' already exists.","traceId":"00-cc8f..."}

{"title":"One or more validation errors occurred.","status":400,
 "errors":{"Name":["The Name field is required."]}}
```

Part 8. Program.cs, the singleton, service changes

src/ProjectHub.Api/Program.cs

```
using System.Text.Json.Serialization;
using ProjectHub.Api.Data;
using ProjectHub.Api.Services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers()
    .AddJsonOptions(options =>
        // Enums as text in JSON: "status": "Todo" instead of "status": 0.
        options.JsonSerializerOptions.Converters.Add(new JsonStringEnumConverter()));
// Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
builder.Services.AddOpenApi();

// ONE shared in-memory store for the whole app: every request gets the same ProjectService object,
// so data created by one request is visible to the next. Topic 7 explains lifetimes (Singleton/Scoped/Transient);
// Topic 10 replaces this with EF Core + SQL Server.
builder.Services.AddSingleton<IProjectService>(_ => new ProjectService(SampleData.CreateProjects()));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

- `AddSingleton<IProjectService>(...)`: **one object for the whole app**. The controller asks for `IProjectService` and gets it.
- **Why a singleton:** the data lives in the service's list. A new `ProjectService` per request would start from the sample data each time: a project created by POST would be gone on the next GET. One shared object used by **parallel** requests is exactly why topic 4's `lock` exists.
- The controller depends on the **interface**: topic 10 swaps in EF Core by changing this one registration. Topic 7 covers the lifetimes (Singleton / Scoped / Transient).

New in ProjectService:

```
public async Task<Project> UpdateAsync(int id, string name, string? description, CancellationToken cancellationToken = default)
{
    var trimmedName = ValidateName(name);

    await SimulateDatabaseCallAsync(cancellationToken);

    lock (_gate)
    {
        var project = FindById(id) ?? throw new NotFoundException(nameof(Project), id);

        // The project may keep its own name; it may not take a name used by ANOTHER project.
        EnsureNameIsFree(trimmedName, exceptProjectId: id);

        project.Name = trimmedName;
        project.Description = description;
        return project;
    }
}

public async Task DeleteAsync(int id, CancellationToken cancellationToken = default)
{
    await SimulateDatabaseCallAsync(cancellationToken);

    lock (_gate)
    {
        var project = FindById(id) ?? throw new NotFoundException(nameof(Project), id);
        _projects.Remove(project);
    }
}

// Call only while holding _gate. The business rule: project names are unique, ignoring case.
private void EnsureNameIsFree(string name, int? exceptProjectId)
{
    if (_projects.Any(p => p.Id != exceptProjectId
                        && string.Equals(p.Name, name, StringComparison.OrdinalIgnoreCase)))
    {
        throw new ConflictException($"A project named '{name}' already exists.");
    }
}
```

- `EnsureNameIsFree(name, exceptProjectId: id)`: a project may keep its own name, not take another's. `ValidateName` and `EnsureNameIsFree` are shared by create and update (one place for the rule).
- `ConflictException : BusinessRuleException`: duplicates now give 409; older code catching `BusinessRuleException` (topic 3/4 demos) still works.
- Changing the interface forced every implementation to change (the topic 3 demo's `LoggingProjectService` got pass-throughs): the compiler checks it.
- **Known limitations:** mapping to DTOs runs outside the lock (a concurrent update could be half-read; EF Core in topic 10 gives each request its own copies); no paging, PATCH, or single-task endpoints yet.

Part 9. What the API really returned

Run against the real API in the cloud container (`dotnet run --project src/ProjectHub.Api --launch-profile http`). The same requests are in `src/ProjectHub.Api/ProjectHub.Api.http` for Rider (run them top to bottom).

Request	Status	Location / body (shortened)
GET all	200 OK	[{"id":1,"name":"Website redesign","description":"New marketing site","createdAt":"2026-09-01T09:00:00Z","task...
GET one	200 OK	{"id":1,"name":"Website redesign","description":"New marketing site","createdAt":"2026-09-01T09:00:00Z","tasks...
GET missing	404 Not Found	{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.5","title":"Not Found","status":404,"traceId":"00-c6...
POST create	201 Created	Location: http://localhost:5064/api/projects/4
POST duplicate (different case)	409 Conflict	{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.10","title":"Conflict","status":409,"detail":"A proj...
POST empty name	400 Bad Request	{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.1","title":"Bad Request","status":400,"detail":"Proj...
POST missing name	400 Bad Request	{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.1","title":"One or more validation errors occurred."...
POST broken JSON	400 Bad Request	{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.1","title":"One or more validation errors occurred."...
PUT update	200 OK	{"id":4,"name":"Docs portal v2","description":"Renamed","createdAt":"2026-10-15T07:36:22.9192103Z","tasks":[]}
PUT to another project's name	409 Conflict	{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.10","title":"Conflict","status":409,"detail":"A proj...
PUT missing	404 Not Found	{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.5","title":"Not Found","status":404,"detail":"Projec...
POST task	201 Created	Location: http://localhost:5064/api/projects/4
POST task to missing project	404 Not Found	{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.5","title":"Not Found","status":404,"detail":"Projec...
DELETE	204 No Content	(empty body)
DELETE again	404 Not Found	{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.5","title":"Not Found","status":404,"detail":"Projec...
GET with non-int id	404 Not Found	(empty body)

Part 10. Debugging the pipeline in Rider

Step	What you did / saw	Meaning
Breakpoint in `Create`, 201 POST	`request` already a record; `cancellationToken.IsCancellationRequested = false`	model binding happened before the action; the client is connected
'no name' POST	breakpoint **not** hit; 400 returned (right)	the framework's `[ApiController]` validation runs **before** the action

Step	What you did / saw	Meaning
`GET /api/projects/abc`	breakpoint not hit; you said 'validated, type not ok' (**wrong reason**)	**routing**: `{id:int}` matches no action; no model binding, no ProblemDetails -> empty 404
Swap the two catches	CS0160 'A previous catch clause already catches all exceptions of this or of a super type' (right)	specific types first
Remove JsonStringEnumConverter	(you asked) `"status": 2`	enums are ints underneath; strings are safer for clients

Part 11. Your answers, corrections, interview Q&A

Question	Your answer	Verdict / complete answer
1. Status codes	200/201/204/400/404/409/500 with examples	mostly right; 404 two flavors; 500 = unexpected failure (the handler only writes it)
2. 400 vs 409	'incorrect shape' vs real conflict	half: 400 = invalid by itself (shape **or** values); 409 = valid but conflicts with current state; retry semantics
3. Idempotent	definition right; 'put/patch' (**wrong**) -> re-check 'get, put, delete' (right)	+ retries after timeouts are safe for PUT/DELETE
4. CreatedAtAction	?? -> re-check: 201 + Location + body (right)	Location = where the new resource lives
5. Why DTOs	no mutable references; no sensitive data exposure (good)	+ over-posting, stable contract, shape per use case
6. [ApiController]	'routing; checking shapes of responses' (half)	checks **requests** (auto 400), ProblemDetails, binding inference; routing comes from [Route]/[HttpGet]
7. CancellationToken	from the caller, cancelled when the client disconnects (mostly)	`HttpContext.RequestAborted`
8. Singleton	skipped -> re-check: one shared unit; otherwise each request is a totally new store (right)	+ parallel requests -> lock
9. /abc -> 404 empty	'not found first of all' (wrong) -> re-check: totally wrong route (right)	route constraint, no action, no ProblemDetails
10. ProblemDetails	standard, useful data, easy to parse (right)	

Interview questions with model answers

Q: What do 200, 201, 204, 400, 404, 409 and 500 mean?

200 OK with body; 201 created (+ Location); 204 success without body; 400 invalid request; 404 resource or route not found; 409 conflicts with current state; 500 unexpected server failure.

Q: 400 vs 409?

400: the request is invalid by itself (retrying never helps). 409: valid, but conflicts with the current state (e.g. the name is taken).

Q: What is idempotency? Which verbs?

Repeating the request leaves the same final state. GET, PUT, DELETE (and HEAD/OPTIONS); not POST; PATCH not necessarily. Matters for safe retries.

Q: What does CreatedAtAction return?

201 Created, a Location header with the new resource's URL (from the named route), and the body.

Q: Why DTOs instead of entities?

Prevent over-posting, keep a stable contract, shape per use case, no leaked references to stored objects, no sensitive data or cycles in JSON.

Q: What does [ApiController] do?

Automatic 400 for invalid models, ProblemDetails error bodies, binding-source inference; requires attribute routing.

Q: ActionResult<T> vs IActionResult?

ActionResult<T> returns either T (200) or another result and documents T for OpenAPI; IActionResult has no typed success value (e.g. Delete).

Q: Where does an action's CancellationToken come from?

HttpContext.RequestAborted; cancelled when the client disconnects. Pass it down to services and EF Core.

Q: Why is the in-memory service a singleton?

The data lives in the service; per-request instances would lose it. One shared instance is used in parallel, hence the lock.

Q: Why does /api/projects/abc return 404 with an empty body?

The {id:int} route constraint matches no action; routing returns 404 before model binding and ProblemDetails.

Q: What is ProblemDetails?

The RFC 9457 standard error body (type, title, status, detail, traceId) so clients handle every error the same way.

Q: How does a service exception become an HTTP status?

It propagates up (through the lock, re-thrown at await) to the controller's typed catch blocks, which map types to codes; unmatched exceptions become 500.

Q: Why must catch (ConflictException) come before catch (BusinessRuleException)?

ConflictException derives from BusinessRuleException; the general catch first would swallow it (the compiler rejects that order).

Q: What does `: ControllerBase` give a controller?

Inheritance: Ok/NotFound/CreatedAtAction/Problem helpers, HttpContext, Request, Response, User, ModelState.

Q: Why can you call Ok() without this.?

Bare names resolve to members of the current object, including inherited ones; this. is implicit (unlike JS).

Q: Why return enums as strings in JSON?

Self-describing, and adding enum values cannot silently shift numeric meanings for existing clients.

Part 12. Summary

Key takeaways

REST: plural nouns in URLs, HTTP methods as verbs; GET/PUT/DELETE idempotent, POST not.
Status codes: 200, 201 (+ Location), 204, 400 (invalid by itself), 404 (resource or route), 409 (conflict with current state), 500 (unexpected).
Pipeline: routing -> model binding -> [ApiController] validation -> action -> result -> JSON.
Controller: `: ControllerBase` (inherited helpers, implicit `this`), primary-constructor injection of `IProjectService`, `ActionResult<T>`, `CreatedAtAction`, `Problem`.
Exceptions: thrown in the service, released through the lock, re-thrown at await, caught by type in the controller -> 404/409/400; anything else -> 500.
DTOs: request and response records; entities never leave the API (over-posting, stable contract, no leaks).
JSON: camelCase, ISO dates, enums as strings, ProblemDetails, automatic 400 for missing required fields.
Singleton: one shared in-memory store across requests, protected by the lock.

Cheat sheet: Express to ASP.NET Core

Express / TS	ASP.NET Core
`router.get('/api/projects/:id', h)`	`[HttpGet("{id:int}")]` on an action in `[Route("api/projects")]`
`req.params.id` / `req.body`	action parameters `int id`, `CreateProjectRequest request` (model binding)
`res.status(200).json(x)`	`return Ok(x);`
`res.status(201).location(url).json(x)`	`return CreatedAtAction(nameof(GetById), new { id }, x);`
`res.sendStatus(204)` / `res.status(404)`	`return NoContent();` / `return NotFound();`
`res.status(409).json({ detail })`	`return Problem(detail: msg, statusCode: 409);`
`req.on('close', ...)` / AbortSignal	`CancellationToken` action parameter
module-level singleton	`builder.Services.AddSingleton<IService>(...)`

Checklist: can you do this without notes?

- Name the status code for each ProjectHub situation and explain 400 vs 409.
- Explain idempotency and which verbs are idempotent.
- Explain what CreatedAtAction returns.
- Give five reasons for DTOs.
- Trace an exception from `EnsureNameIsFree` to a 409 response.
- Explain why the service is a singleton and why that needs a lock.
- Explain why `/api/projects/abc` gives an empty 404.
- Read `ProjectsController(IProjectService projects) : ControllerBase` aloud and explain both parts.

What comes next: Topic 7 (DI, middleware, config, ILogger)

Dependency injection properly: Singleton vs Scoped vs Transient and when each is right; writing our own **middleware** (request timing, like an Express middleware); configuration from `appsettings.json` (the simulated latency becomes a setting); and `ILogger` for structured logs.